SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

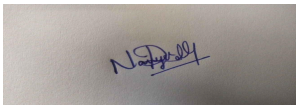
CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

Code of Conduct:

I NARRAVULA NAVYA SRI - 192324220 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

1.1 Identify incorrect or invalid fields

- Status line (`HTTP/1.1 200 OK`): Syntactically valid, but the status may be inappropriate if the server failed to generate a valid response.
- Date: The weekday is incorrect. April 1, 2026, is a Wednesday, not a Monday.
- Content-Type: Valid. It specifies that the response body contains HTML.
- Content-Length: Invalid because a negative value is not permitted.
- Connection: Valid. `keep-alive` indicates that the connection may remain open for additional requests.

1.2 Issue with Content-Length and its impact

The `Content-Length` header specifies the size of the response body in bytes. Its value must be a non-negative integer.

Here, `Content-Length: -150` is invalid.

Possible impacts:

- The server or client may reject the malformed header.
- Different HTTP implementations may handle the invalid value differently.
- Incorrect message framing can cause parsing errors or connection problems if the message length is otherwise misreported.

1.3 Does the status code match the response correctness?

`200 OK` indicates that the request was successfully handled. However, the response contains an invalid header value.

The status code alone does not guarantee that every header is valid. Therefore, a `200 OK` response can still contain malformed metadata.

1.4 Corrected response

Assuming the response body is 150 bytes long:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

The `Content-Length` value must match the actual number of bytes in the response body. If the body has a different size, use that size instead.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

2.1 Identify missing or improperly closed elements

The paragraph element `<p>` is missing its closing tag, `</p>`.

The other elements shown have matching closing tags.

2.2 How will the browser interpret this structure?

Browsers use HTML parsing rules to recover from common markup errors. When the parser encounters `</div>` while a paragraph is open, it implicitly closes the paragraph before closing the `<div>`.

The resulting DOM will effectively resemble:

```
  <p>Hello World</p>

</div>
```

Thus, the browser will usually render the content instead of displaying a parsing error.

2.3 Impact on rendering and layout

- The text `Hello World` will normally appear inside the paragraph.
- The paragraph's default margins will generally be applied.
- The browser may automatically repair malformed markup.
- More complicated markup errors can produce unexpected nesting or layout differences across browsers.

In this example, the visual impact is likely minimal because the browser can recover automatically.

2.4 Corrected DOM structure

```
<!DOCTYPE html>

<html>

  <head>

    <title>Test Page</title>

  </head>

  <body>

    <div>

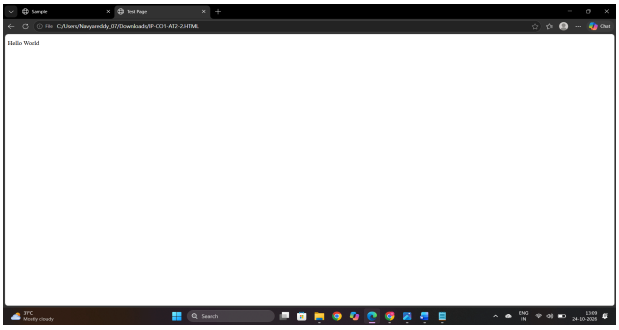
      <p>Hello World</p>

    </div>

  </body>

</html>
```

Adding the closing `</p>` makes the intended structure explicit.



3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

3.1 Differences between GET and POST

Feature	GET	POST
Data transmission	Usually included in the URL query string	Usually included in the request body
Visibility	Parameters can appear in URLs, browser history, and logs	Body is not normally shown in the URL
Caching	May be cached	Not normally cached by default
Bookmarking	URLs with parameters can be bookmarked	Request body is not stored in a bookmark
Typical purpose	Retrieving resources	Submitting data or performing operations
Suitability for passwords	Unsuitable	Preferred, when combined with other security controls

3.2 Which method is more suitable for login?

POST is more suitable for login authentication because credentials are sent in the request body rather than the URL.

However, POST alone does not encrypt or protect credentials. HTTPS is essential to protect them during transmission.

3.3 Security risks of using GET for authentication

Using GET to transmit usernames and passwords can expose credentials through:

1. Browser history: URLs may remain in browsing history.
2. Server logs: Web servers and monitoring systems may record complete URLs.
3. Shared links: URLs containing credentials can be copied or shared accidentally.
4. Referrer headers: URLs may leak to other sites in certain circumstances.
5. Caching: Sensitive URLs may be retained by caches or other intermediaries.

These risks make GET inappropriate for transmitting passwords.

3.4 Best practices for secure login implementation

- Use HTTPS across the entire authentication process.
- Submit credentials using POST in the request body.
- Never store passwords in URLs, logs, or client-side storage.
- Store passwords using a strong password-hashing algorithm such as Argon2id or bcrypt, with appropriate parameters and unique salts.
- Use secure session cookies with `HttpOnly`, `Secure`, and an appropriate `SameSite` setting.
- Implement rate limiting and protections against brute-force attacks.
- Use multi-factor authentication (MFA) when appropriate.
- Validate input on the server and return generic authentication errors.
- Protect authenticated sessions against session fixation and other session attacks.

Conclusion: POST over HTTPS, combined with secure password storage and session management, is the appropriate approach for login systems.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

4.1 Identify missing closing tags

The following closing tags are missing:

- `</h1>` — closes the heading.
- `</p>` — closes the paragraph.

The `<img>` element is a void element in HTML, so it does not require a closing tag.

4.2 Analyze issues with the <img> tag

The image tag is valid in its basic form:

```
<img src="image.jpg">
```

However, it has two potential issues:

- Missing alternative text: The `alt` attribute should describe the image for accessibility. If the image is purely decorative, use `alt=""`.
- Image path: The file `image.jpg` must exist at the specified relative location, or the browser will display a broken-image indicator.

4.3 How will browsers handle these errors?

Browsers generally recover from missing closing tags using their HTML parsing rules.

- The heading will normally be closed implicitly when the parser encounters the paragraph element.
- The paragraph will be closed when the parser reaches the end of the document's body content.
- The image will load if the file is available and accessible.
- Missing alternative text may reduce accessibility for users relying on screen readers.

Although browsers can repair many errors, explicitly closing elements improves readability and maintainability.

4.4 Corrected HTML code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Sample</title>

</head>

<body>

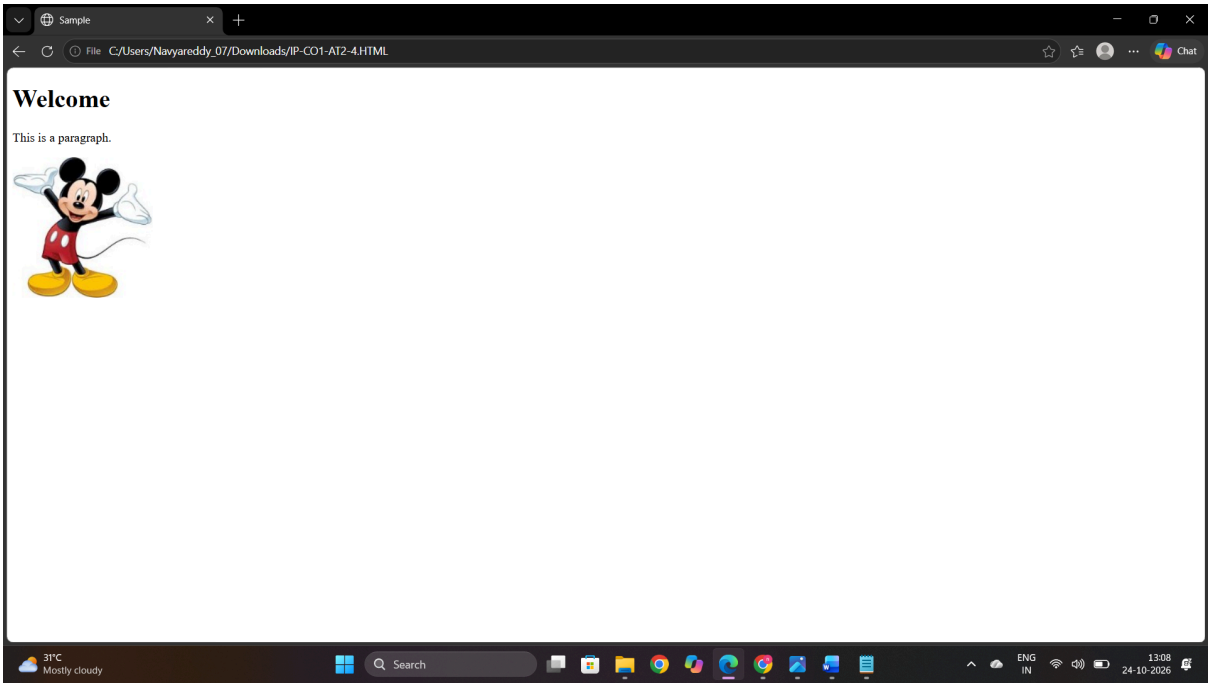
  <h1>Welcome</h1>

  <p>This is a paragraph.</p>

  <img src="image.jpg" alt="Description of the image">

</body>

</html>
```



Replace `"Description of the image"` with an accurate description of the actual image. The added `lang`, character encoding, and viewport declarations also improve accessibility and mobile compatibility.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

5.1 Explain each step in the communication flow

Step 1: User enters a URL

The browser parses the URL to identify the protocol, domain name, path, and any query parameters. It may also check its cache or existing connections.

Step 2: DNS lookup

The browser or operating system resolves the domain name, such as example.com, to an IP address. A cached DNS result may eliminate the need for a new lookup.

Step 3: HTTP request is sent

The browser establishes or reuses a network connection and sends an HTTP request containing a method, path, headers, and, when applicable, a request body.

Step 4: Server processes the request

The server receives the request, performs necessary application logic, may query a database or another service, and prepares a response.

Step 5: Response is returned

The server sends an HTTP status code, response headers, and possibly a response body. The response may contain HTML, JSON, images, or other resources.

Step 6: Browser renders the page

The browser parses HTML into the DOM, processes CSS, executes JavaScript, loads dependent resources, and paints the resulting page on the screen.

5.2 Where can delays occur?

Delays can happen at several points:

- DNS resolution: Slow DNS servers or cache misses.
- Connection establishment: TCP connection setup and TLS negotiation for HTTPS.
- Network transmission: High latency, congestion, packet loss, or slow connections.
- Server processing: Expensive application logic or slow database queries.
- Response transfer: Large HTML documents, images, scripts, or other resources.
- Browser rendering: Complex DOM trees, large stylesheets, JavaScript execution, and layout recalculations.

A page may also take longer to become interactive if JavaScript blocks the main thread or required resources load slowly.

5.3 Role of DNS

DNS (Domain Name System) translates human-readable domain names into IP addresses that computers use to communicate.

For example:

`www.example.com` → an IP address associated with the domain.

DNS enables users to access websites without memorizing numerical IP addresses. It also supports features such as multiple records, load distribution, and routing to appropriate services.

DNS resolution may involve a browser or operating-system cache, a recursive resolver, and authoritative DNS servers. If a valid cached result exists, the lookup can be much faster.

5.4 Optimizations to improve performance

Area	Optimization	Benefit
DNS	Use reliable DNS services and appropriate caching	Reduces lookup latency
Network	Use HTTP/2 or HTTP/3 where supported	Improves connection and request efficiency
Security	Reuse HTTPS connections	Avoids repeated connection setup
Server	Optimize application code and database queries	Reduces server response time
Content delivery	Use a CDN for static assets	Reduces distance between users and content
File size	Compress text resources with Brotli or gzip	Reduces transfer time
Images	Resize and compress images; use WebP or AVIF when suitable	Reduces page weight
Browser	Cache static resources with suitable cache headers	Avoids unnecessary downloads
JavaScript	Defer noncritical scripts and reduce unused code	Improves rendering and interactivity
Rendering	Inline critical CSS where appropriate and minimize render-blocking resources	Speeds up initial page display

Conclusion: Website performance improves when DNS resolution, network connections, server processing, resource delivery, and browser rendering are optimized together.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided

Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand